

Workflow and Agents

Sep 30, 2026 · @Salike Hassan

Workflow = code defines the steps and their order in advance. **Agent** = Claude decides the next step itself, using tools, until it judges the goal is done.

1. Why we need workflows and agents: fit first, then choose

My way. Whenever we use any AI capability, the first step is to check whether a workflow or an agent really fits the problem at all. Only after that comes the second question: which one to pick and which to leave. The 4D framework from AI Fluency (Delegation, Description, Discernment, Diligence) should always be in front of us when making that choice.

4D	How I use it to choose
Delegation	What can Claude do here, and does the problem fit a workflow or an agent at all?
Description	Do I know exactly what instruction to give Claude?
Discernment	Do I know what the output should look like and how to judge it?
Diligence	Diligence is our responsibility for the AI's work: verifying that the output is reliable, not just plausible-looking. The evaluator-optimizer pattern and the agent loop's "verify" phase build that diligence into the architecture.

When all four are clear (I know the steps, the instructions, the expected output and how it is checked), I can go with a **workflow**.

CCA tip. Anthropic's guidance is to start with the simplest solution (a single well-prompted call), move to a workflow only when a fixed pattern clearly fits, and use an agent only when the task's unpredictability genuinely needs Claude to control the flow. More autonomy is not automatically better.

2. Workflow vs agent

My way. As an engineer, when I know the problem needs close to 100% accuracy and I know the steps, my preferred choice is a **workflow**. When the problem is abstract, I accept some anomaly and cannot promise 100% accuracy, and I do not know exactly what needs to be done, an **agent** fits. Agents also make the UI and user experience more flexible than a workflow.

Aspect	Workflow	Agent
Control	Code defines fixed steps in advance	Claude decides what to do next, dynamically
Steps	Known number of steps, in a known order	Number of steps unknown upfront; the path is discovered during execution
Predictability	High: same steps every time	Lower: steps vary with Claude's judgment
Accuracy	Can aim for the highest accuracy	May not reach 100%; flexibility matters more
Cost / latency	Bounded and predictable	Variable: depends on how many loop iterations Claude takes
Example	RAG pipeline: retrieve, augment, generate	Agentic RAG: Claude decides to re-query or stop

Addition. The number of steps is a key test. If it is known, use a workflow. If the path can only be found while running, use an agent.

3. The five workflow patterns

All five are workflows because code controls the flow. The choice of pattern depends on what exactly we are trying to solve and how.

Pattern	How it works	Best for
Prompt chaining	Output of one call feeds the next, in a fixed sequence	Tasks where each step depends on the previous one
Routing	A first call classifies the input, then code sends it to a specialised prompt	Distinct input categories, each needing its own prompt
Parallelization	Independent calls run at the same time and the results are aggregated (sectioning or voting)	Independent subtasks, or higher confidence through redundancy
Orchestrator-workers	A lead call splits the task into subtasks at runtime; worker calls handle them; results are recombined	Tasks where the number of subtasks is unknown upfront
Evaluator-optimizer	One call generates, another evaluates; the critique feeds back until it passes or a max-iteration limit	Output that must meet clear quality criteria

3.1 My three patterns, with my examples

My way: parallel. The same problem is pushed into different categories, each gives its own result, and we aggregate them into one response. Example (from Anthropic): I upload an image of a material and want output to feed into a CAD tool that decides between metal, plastic or polymer. Three or four parallel prompts each judge the image against a defined set of physical requirements, then one more prompt evaluates all the results and gives the final recommendation.

My way: chaining. Each step depends on the previous step, so the steps run one after another.

Addition: chaining in detail. The output of one Claude call becomes the input to the next, in a fixed sequence. Each step does one well-defined transformation, and the quality of each step gates the next. Because every call has one narrow job, each prompt stays simple and the result is easier to check.

```
draft = call_claude(prompt="Write a project update draft")
polished = call_claude(prompt=f"Tighten this for executives: {draft}")
final = call_claude(prompt=f"Add a one-line TL;DR: {polished}")
```

Chaining	Detail
Control	Code fixes the order of the steps; Claude does not choose the next step
Best for	Tasks that split cleanly into sequential sub-tasks where each step needs the previous output
Check between steps	Code can validate one step's output before passing it on, and stop early if it fails
Trade-off	Steps run one after another, so it is slower than parallel calls; each call is simpler and more accurate
Vs parallel	Parallel steps are independent; chained steps depend on each other

My way: routing. I post videos on social media in categories such as education, comedy, travel and programming. Based on today's trending topic, the routing step picks the category and sends the topic to the matching prompt.

Addition. Parallelization has two variants. **Sectioning** splits a task into independent parts (my materials example). **Voting** runs the same task several times and takes a majority or consensus, for higher confidence.

Correction. The LLM called Orchestrator-Subagent a "fourth pattern". It is **not an extra pattern: it is Orchestrator-workers, one of the five (the fourth in Anthropic's list)**. Evaluator-optimizer is the fifth. I had covered only parallel, chaining and routing.

Insight. In sectioning, code decides the subtasks up front. In orchestrator-workers, Claude decides them at runtime. That makes it a workflow with an agent-like decision in its first step, which is why the line between workflow and agent is a spectrum, not a hard boundary. It still counts as a workflow because a fixed code loop runs delegate, collect, recombine.

Insight. Evaluator-optimizer is model-based grading turned into a live, in-the-loop pattern: the same grading, running during generation instead of afterwards in a test suite.

Analogy. Routing is like support-ticket triage: one classifier reads the ticket and sends it to billing, technical or account access, each handled by a specialist with a narrower prompt.

CCA tip. Know all five by name: Prompt Chaining, Routing, Parallelization (sectioning + voting), Orchestrator-Workers, Evaluator-Optimizer.

4. Agents and the agent loop

My way. For an agent we give Claude an abstract idea of the goal and, optionally, some tools. Then we let Claude decide which tool it needs to solve the problem. We may not reach 100% accuracy, but the agent works on its own, which makes the experience more flexible than a workflow.

4.1 How the loop works

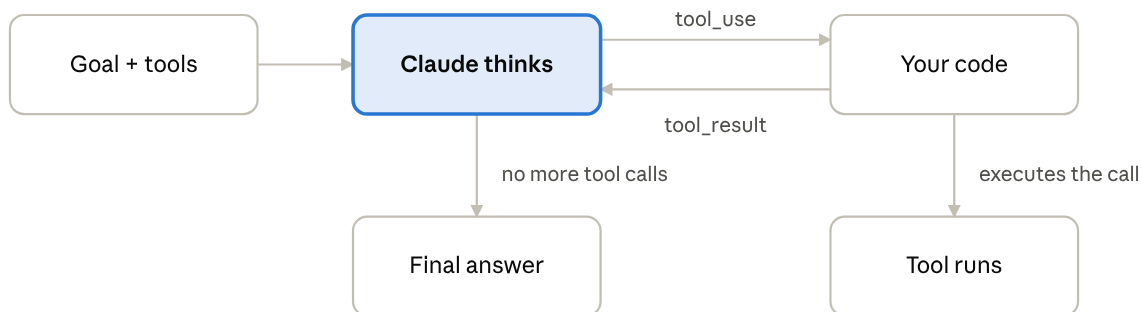
1. The user gives Claude a goal and the available tools.
2. Claude thinks about what to do next.
3. Claude decides to use a tool and returns a **tool call** with parameters (a `tool_use` block).
4. **Our code** executes the tool.
5. Our code sends the result back to Claude (a `tool_result` block).
6. Claude thinks again based on the result: it either calls another tool (back to step 3) or decides it is done.

7. The final response is returned to the user.

CCA tip. Claude never executes tools itself. It only **requests** them; your code always runs the tool and returns the result.

The loop shape from my earlier notes is the same: gather context, take action, verify the work, repeat.

Claude requests tools, your code runs them



Repeats until Claude is done or your max_turns limit is hit

agent loop · Claude requests, your code executes

Claude only asks for a tool call; your code runs it and sends the result back, and the loop repeats until Claude is done.

4.2 When the loop stops

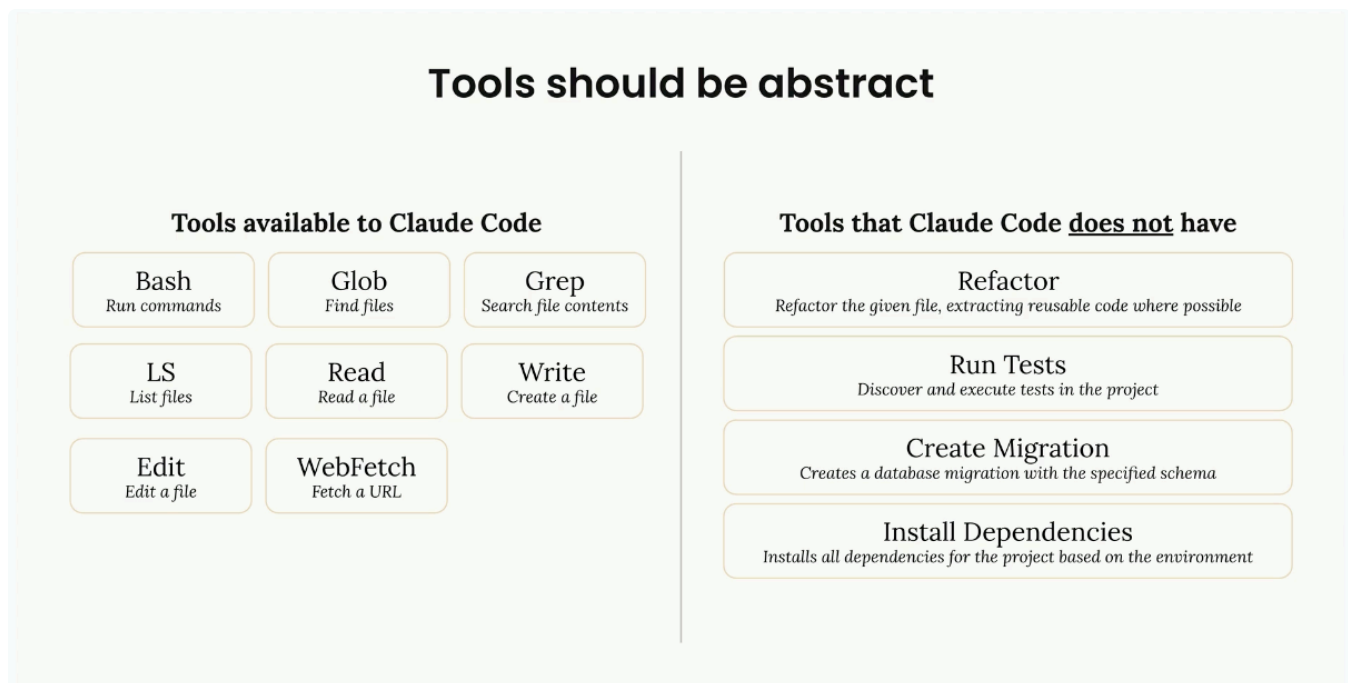
- Claude judges the task complete and answers with no further tool calls.
- A hard `max_turns` limit set by our code is reached, whether or not Claude thinks it is done.
- The run ends with an error (for example `error_max_turns` or `error_during_execution`).

Addition. Agents are also the right choice when the number of steps is unknown upfront. Workflows suit a known number of steps in a known order; agents suit a path that is discovered while running.

Correction. My social media routing example is actually a **workflow**, because the routing rules are predefined by me. The agent version would give Claude a trending-topics API, my content library and my posting history, and let it decide on its own what to post, when and where, with no routing rules from me.

Insight. Sub-agents exist because of context-window limits. Giving each sub-agent a fresh context and only the tools it needs manages context and limits permissions at the same time. Effort level (low, medium, high) and lifecycle hooks are two other controls on top of the same loop.

4.3 Tools for an agent should be abstract



Claude Code is built from a few general tools (Bash, Read, Edit and so on) rather than one tool per task like Refactor or Run Tests, because Claude can combine general tools to do those tasks itself.

Best practice:
Provide reasonably abstract tools that Claude can combine together

Tools for a Social Media Video Agent

Name	Description
bash	Provides access to FFMPEG, a CLI tool that can stitch together images and sound to form a video
generate_image	Generate an image based upon a prompt
text_to_speech	Converts written text into spoken audio
post_media	Posts text, images, or videos to a social media account

The same idea for a social media video agent: four general tools (bash with FFMPEG, generate_image, text_to_speech, post_media) are enough for Claude to build and post a video by combining them.

Marketing Agent

You

Create and post a video on Python programming

AI

OK, I will create and post the video

Send

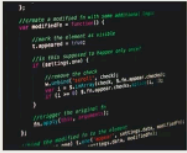
Marketing Agent

You

I want you to make a video on Python, but first I want to pick out an initial image for the video.

AI

Sure! Does this one work?



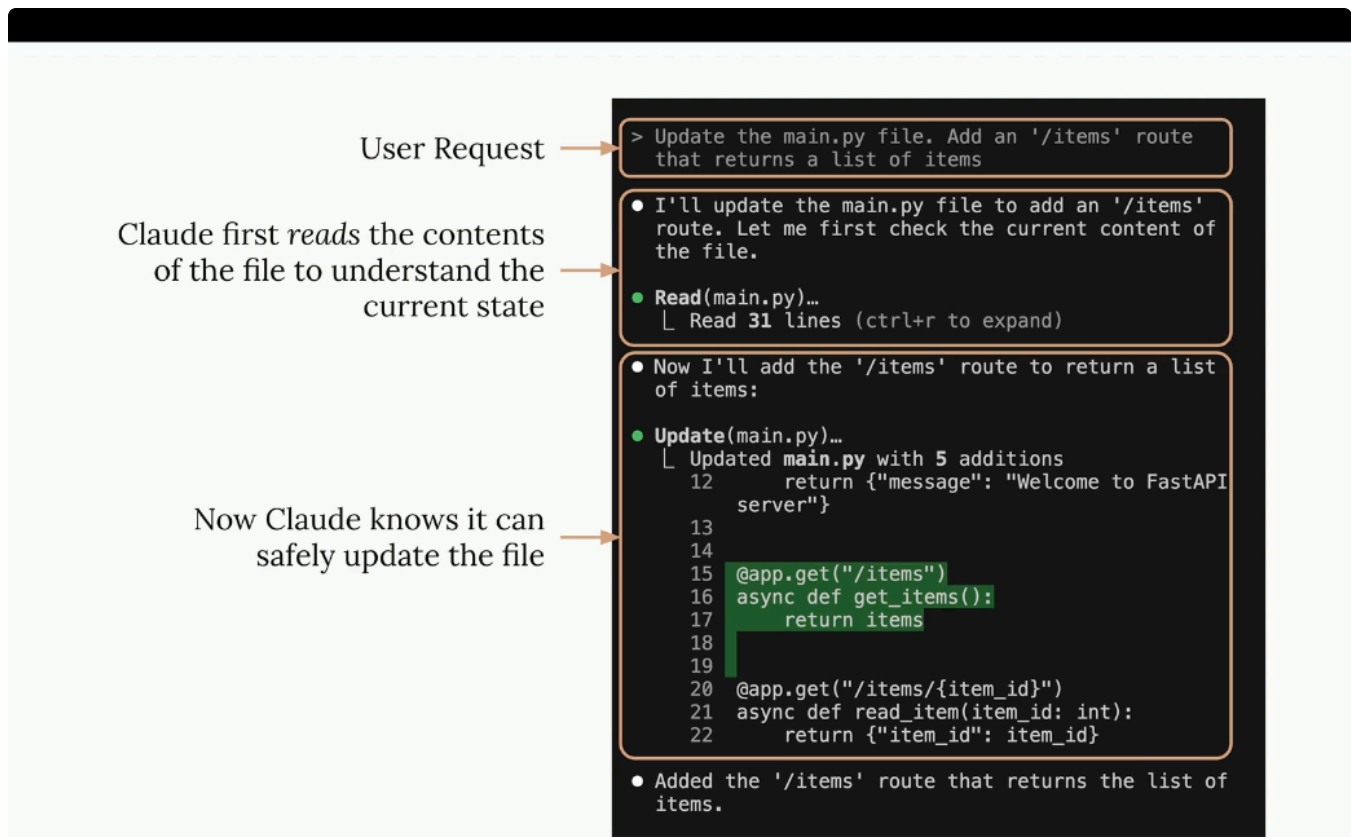
Send

Because the tools are general rather than a fixed pipeline, the agent can follow the user's changing request, such as pausing to get an image approved before continuing.

Insight. This is the workflow-vs-agent difference again: a workflow hard-codes "create and post a video" as fixed steps, while an agent with abstract tools can handle either request in the screenshot using the same four tools.

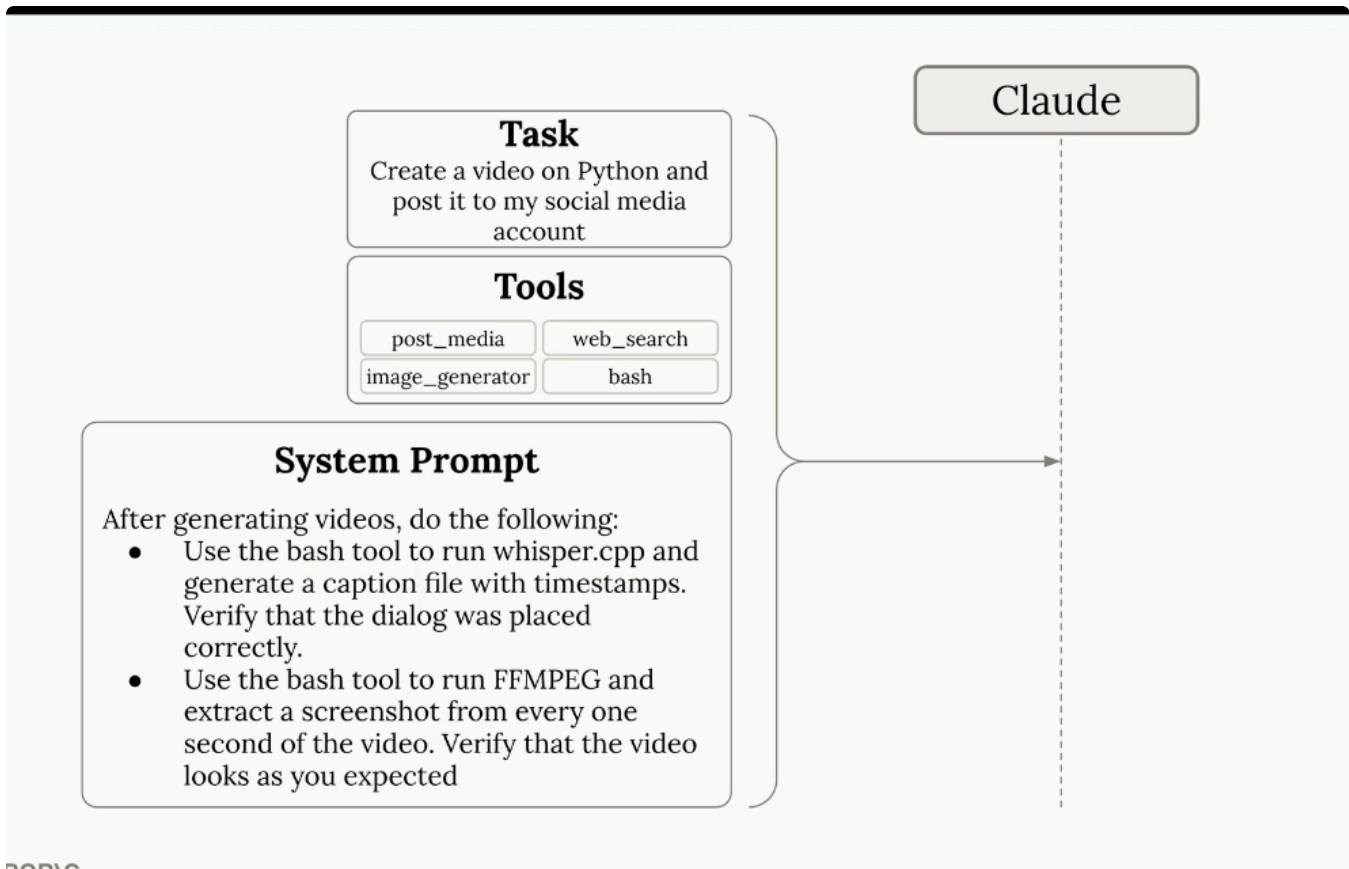
5. Environment inspection

My way. For an agent to start, progress and complete a piece of work, it needs to be aware of its environment. Example: when I ask Claude Code to add a new endpoint, it first uses its file-reading tools to understand the current state of the file, and if needed it asks a follow-up question. Then it uses the edit tool to add the endpoint. So an agent uses different tools and prompt instructions to understand the current state of the environment. Claude is effectively blind, and it uses environment inspection to start, progress and complete the work.



In the screenshot, Claude reads the 31 lines of `main.py` first, and only then edits it, because now it knows the file's current state and can update it safely.

When	What the agent inspects	Example
Start	The current state, before changing anything	Read the file; ask the user a follow-up question if something is unclear
Progress	The result of each action, to decide the next step	Look at the tool result after each edit before choosing the next tool
Complete	Whether the work actually came out as expected	Check the final output instead of assuming it is right



This screenshot shows the "complete" part: after generating a video, the system prompt tells Claude to run `whisper.cpp` to produce a timestamped caption file and check the dialog is placed correctly, and to run `FFMPEG` to extract a screenshot every second and check the video looks as expected. Claude cannot see the video itself, so it inspects it through tools.

Addition. The agent has three ways to inspect its environment: **tools** (read, search and list files, run commands), **prompt instructions** (the system prompt says what to check and when), and **follow-up questions** to the user.

Insight. This is the "gather context" and "verify work" parts of the agent loop in section 4, and it is Diligence from the 4D framework built into the agent: instead of assuming its action worked, the agent checks.

6. Choosing between workflow and agent

Ask these in order:

1. Does this problem fit an AI workflow or agent at all? If a single well-prompted call solves it, stop there.
2. Do I know the steps, the instructions and the expected output (the 4D check in section 1)? If yes, use a **workflow**, and pick the pattern by the shape of the problem:
 - steps that depend on each other: chaining
 - different input categories: routing
 - independent parts or redundancy: parallelization
 - subtasks unknown until runtime: orchestrator-workers
 - a clear quality bar to iterate towards: evaluator-optimizer
3. Is the problem ambiguous, with an unknown number of steps, and can I accept some anomaly in exchange for flexibility? Then use an **agent** with a goal and tools, and set a stopping condition such as `max_turns`.

Insight. This is the 4D framework applied to agents. Delegation is deciding whether the task needs an agent (dynamic judgment) or a workflow (fixed steps). Description is the system prompt and tool definitions I hand over. Discernment includes giving each sub-agent only the tools it needs. Diligence is verifying the output is reliable, which the evaluator-optimizer pattern and the loop's verify phase build into the architecture.

Tech note. Orchestrator-workers sits between the two: Claude decides the subtasks, but a fixed code loop still controls delegate, collect and recombine. A lead Claude coordinating specialist Claude instances is the pattern to recognise in multi-domain tasks.

7. One-liner and quick revision

One-liner. Workflows are best when the steps are known, the order is predictable and accuracy must be high: chaining for dependent steps, routing for conditional branching, parallelization for independent tasks, orchestrator-workers when subtasks emerge at runtime, evaluator-optimizer to iterate to a quality bar. Agents are best when the problem is ambiguous, the number of steps is unknown and flexibility matters more than guaranteed accuracy: Claude picks tools and order through a loop of think, act, observe, repeat, while your code runs the tools.

Term	Remember
Workflow	Code controls the flow
Agent	Claude controls the flow, with tools
4D	Delegation, Description, Discernment, Diligence
Prompt chaining	Output of one call feeds the next
Routing	Classifier call sends input to a specialised prompt
Parallelization	Sectioning (split) or voting (repeat)
Orchestrator-workers	Lead call assigns subtasks at runtime
Evaluator-optimizer	Generate, evaluate, refine until it passes
Agent loop	Think, tool call, your code runs it, result back, repeat
max_turns	Hard code-enforced cap on loop iterations
Sub-agent	Own context window and limited tool permissions